

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D040.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 040 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

matplotlib 量化作图

// Matplotlib

KEY CONCEPTS · 三大要点

- ▶ axes 系统
- ▶ subplots
- ▶ 样式

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

一张能讲清楚故事的图,胜过一堆数字。这一节用真实 A 股,带你把 matplotlib 的核心套路一次打通:plt 偷懒风格和 ax 面向对象风格的区别、subplots 多面板布局、twinx 双 y 轴、mdates 日期格式化,最后亲手画出一张净值、回撤、均线信号、成交量四合一的专业量化分析图。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 分清 plt 偷懒风格和 fig/ax 面向对象风格,知道多图时必须用 ax
- 02 会用 subplots 把价格、成交量、回撤分面板上下排布,共享同一条时间轴
- 03 会用 twinx 在一张图里放两个量纲不同的 y 轴(如价格和成交量)
- 04 会用 mdates 把横轴日期格式化得清爽专业
- 05 能把净值、最大回撤、金叉死叉信号叠加成一张完整的量化分析图

02 · CONTEXT · 历史与背景

小钱的净值图被导师批"看不懂",一周后脱胎换骨

// 不读历史的策略走不远

小钱第一次给导师汇报策略,甩出一张图:横轴是一串挤成黑团的日期,价格和成交量硬塞在一个 y 轴上,成交量几百万的大数字把价格曲线压成了一条贴着底的直线,根本看不出走势。导师只说了四个字:看不懂,重画。小钱回去恶补了一周 matplotlib,学会了用 subplots 把价格、成交量、回撤分成三个面板上下对齐,用 twinx 给量纲差很多的两个量各放一个 y 轴,用 mdates 把日期格式化清爽的年月,还在净值曲线上标出金叉死叉的买卖点。再汇报时,导师盯着图看了半天,说了句:这才像个做量化的。这一节我们就把小钱学会的这套作图本事,一次教给你。

03 · CORE CONCEPTS · 核心概念详解

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

plt 风格 vs ax 风格

plt.plot 是偷懒风格,直接在"当前图"上画,快速看一眼够用;fig, ax = plt.subplots() 是面向对象风格,每个 ax 是1 块独立的画框,想画哪块就对哪个 ax 下命令,多子图布局必须用它。

CONCEPT_02

subplots 多面板

一句话开出几行几列的画框网格,比如三行一列,把价格、成交量、回撤分别画在3 块上下排好的画框里,设成共享 x 轴,时间就对得整整齐齐。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

twinx 双 y 轴

当两个量的量纲差很多(价格几百、成交量几百万),硬画在一个 y 轴上小的那个会被压扁。
twinx 在同1块画框上叠一个共享 x 轴的第二个 y 轴,左右各管一个量。

CONCEPT_04

mdates 日期格式化

时间轴默认会把日期挤成一团。mdates 能指定多久标一个刻度(比如每3个月)、用什么格式显示(比如年-月),让横轴清爽好读。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

中文字体与负号

matplotlib 默认不显示中文,会变成方块,负号也可能显示异常。需要指定中文字体并关掉 unicode 负号,这是中文量化作图的标准开场代码。

04 · PYTHON LAB · 真实可跑代码

matplotlib 五件套 — plt vs ax / subplots 多面板 / twinx 双轴 / mdates 日期 / 信号叠加净值回撤图

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_040_matplotlib.py - matplotlib 量化作图:plt vs ax / subplots 多图 / twinx 双
轴 / mdates 日期 / 中文字体
# 把净值、回撤、均线信号、成交量画成一张专业的量化分析图
# 数据:baostock 日线(免费、稳定、国内零翻墙)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import baostock as bs

pd.set_option('display.width', 140)
CODE, NAME = 'sh.600519', '茅台'
START, END = '2022-01-01', '2024-12-31'

print('==== 0. 数据拉取(baostock 日线 收盘+成交量)====')
lg = bs.login()
if lg.error_code != '0':
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
rs = bs.query_history_k_data_plus(CODE, 'date,close,volume', start_date=START,
end_date=END,
frequency='d', adjustflag='2')
rows = []
while rs.error_code == '0' and rs.next():
    rows.append(rs.get_row_data())
bs.logout()
df = pd.DataFrame(rows, columns=['date', 'close', 'volume'])
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/5)

matplotlib 五件套 — plt vs ax / subplots 多面板 / twinx 双轴 / mdates 日期 / 信号叠加净值回撤图

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
df['date'] = pd.to_datetime(df['date'])
df['close'] = df['close'].astype(float)
df['volume'] = df['volume'].astype(float)
df = df.set_index('date').sort_index()
close, vol = df['close'], df['volume']
print(f'{NAME} 日线 {len(close)} 条 {close.index[0].date()} → {close.index[-1].date()}')

# ===== 准备要画的量:净值 / 均线 / 回撤 / 信号 =====
print('\n==== 1. 准备绘图数据(净值/均线/回撤/金叉死叉信号)====')
nv = close / close.iloc[0] # 净值(起点 1)
ma20 = close.rolling(20).mean()
ma60 = close.rolling(60).mean()
peak = nv.expanding().max()
dd = nv / peak - 1 # 回撤(<=0)
max_dd = dd.min()
# 金叉/死叉:ma20 上穿/下穿 ma60
cross = (ma20 > ma60).astype(int).diff()
golden = close.index[cross == 1] # 金叉日
death = close.index[cross == -1] # 死叉日
print(f'净值期末 {nv.iloc[-1]:.2f} 倍,最大回撤 {max_dd*100:.1f}%(谷底 {dd.idxmin().date()}')
print(f'金叉 {len(golden)} 次,死叉 {len(death)} 次')

# ===== 2. plt vs ax 风格说明(打印,概念在视频讲)=====
print('\n==== 2. plt 风格 vs ax 风格 ====')
print('plt.plot(...) 偷懒风格:画一张简单图够用,但多子图时分不清画在哪')
print('fig, ax = plt.subplots():面向对象风格,每个 ax 是一块独立画框,多图布局必须用它')

# ===== 3. subplots 多面板:价格+均线+信号 / 成交量 / 回撤 =====
print('\n==== 3. subplots 三面板 + twinx + mdates ====')
fig, axes = plt.subplots(3, 1, figsize=(13, 10), sharex=True,
```


04 · PYTHON LAB · 真实可跑代码 · 续 (3/5)

matplotlib 五件套 — plt vs ax / subplots 多面板 / twinx 双轴 / mdates 日期 / 信号叠加净值回撤图

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
gridspec_kw={'height_ratios': [3, 1, 1.3]})
ax_p, ax_v, ax_d = axes
plt.rcParams['axes.unicode_minus'] = False

# 面板1:价格 + 双均线 + 金叉死叉信号
ax_p.plot(close.index, close, color='#4c566a', lw=1.0, label=f'{NAME} 收盘')
ax_p.plot(ma20.index, ma20, color='#d08770', lw=1.4, label='20 日均线')
ax_p.plot(ma60.index, ma60, color='#5e81ac', lw=1.4, label='60 日均线')
ax_p.scatter(golden, close.reindex(golden), marker='^', s=90, color='#bf616a',
             zorder=5, label='金叉')
ax_p.scatter(death, close.reindex(death), marker='v', s=90, color='#a3be8c',
             zorder=5, label='死叉')
ax_p.set_title(f'{NAME} 量化分析图:价格+双均线+信号 / 成交量 / 回撤', fontsize=14)
ax_p.set_ylabel('价格(元)'); ax_p.legend(loc='upper left', ncol=2)

# 面板2:成交量(柱)+ twinx 叠加净值
ax_v.bar(vol.index, vol / 1e6, color='#b48ead', width=1.0, alpha=0.6)
ax_v.set_ylabel('成交量(百万股)', color='#b48ead')
ax_v2 = ax_v.twinx() # twinx:共享 x 轴的第二个 y 轴
ax_v2.plot(nv.index, nv, color='#ebcb8b', lw=1.2)
ax_v2.set_ylabel('净值', color='#ebcb8b')

# 面板3:回撤(填充)
ax_d.fill_between(dd.index, dd * 100, 0, color='#bf616a', alpha=0.5)
ax_d.set_ylabel('回撤 %'); ax_d.axhline(max_dd*100, color='#bf616a', ls='--',
    lw=0.8)
ax_d.text(dd.idxmin(), max_dd*100, f' 最大回撤 {max_dd*100:.0f}%', va='bottom',
    color='#bf616a')

# mdates:把横轴日期格式化成 年-月
ax_d.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
```

```
ax_d.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))  
plt.setp(ax_d.get_xticklabels(), rotation=30, ha='right')  
plt.tight_layout(); plt.savefig('chart_01.png', dpi=110); plt.close()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/5)

matplotlib 五件套 — plt vs ax / subplots 多面板 / twinx 双轴 / mdates 日期 / 信号叠加净值回撤图

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('图1(三面板组合图)已保存')

# ===== 4. 对比:偷懒 plt 风格画一张简单价格图 =====
fig = plt.figure(figsize=(11, 4.5))
plt.plot(close.index, close, color='#5e81ac')
plt.title(f'{NAME} 收盘价(plt 偷懒风格,适合快速看一眼)')
plt.ylabel('价格(元)')
plt.tight_layout(); plt.savefig('chart_02.png', dpi=110); plt.close()
print('图2(plt 偷懒风格单图)已保存')

# ===== 5. twinx 单独演示:价格 vs 成交量双 y 轴 =====
fig, ax1 = plt.subplots(figsize=(12, 5))
ax1.plot(close.index, close, color='#5e81ac', lw=1.1)
ax1.set_ylabel('收盘价(元)', color='#5e81ac'); ax1.tick_params(axis='y',
labelcolor='#5e81ac')
ax2 = ax1.twinx()
ax2.bar(vol.index, vol / 1e6, color='#d08770', alpha=0.35, width=1.0)
ax2.set_ylabel('成交量(百万股)', color='#d08770'); ax2.tick_params(axis='y',
labelcolor='#d08770')
ax1.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
ax1.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
plt.setp(ax1.get_xticklabels(), rotation=30, ha='right')
ax1.set_title(f'{NAME} 价格 + 成交量双 y 轴(twinx)')
plt.tight_layout(); plt.savefig('chart_03.png', dpi=110); plt.close()
print('图3(twinx 双轴)已保存')

# ===== 6. 信号叠加净值图 =====
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(nv.index, nv, color='#4c566a', lw=1.1, label='净值')
ax.fill_between(dd.index, 1+dd, nv, color='#bf616a', alpha=0.12)
ax.scatter(golden, nv.reindex(golden), marker='^', s=80, color='#bf616a',
```

```
zorder=5, label='金叉')
```

```
ax.scatter(death, nv.reindex(death), marker='v', s=80, color='#a3be8c', zorder=5,  
label='死叉')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/5)

matplotlib 五件套 — plt vs ax / subplots 多面板 / twinx 双轴 / mdates 日期 / 信号叠加净值回撤图

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.set_title(f'{NAME} 净值 + 金叉死叉信号叠加'); ax.set_ylabel('净值'); ax.legend()
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
plt.setp(ax.get_xticklabels(), rotation=30, ha='right')
plt.tight_layout(); plt.savefig('chart_04.png', dpi=110); plt.close()
print('图4(信号叠加净值)已保存')

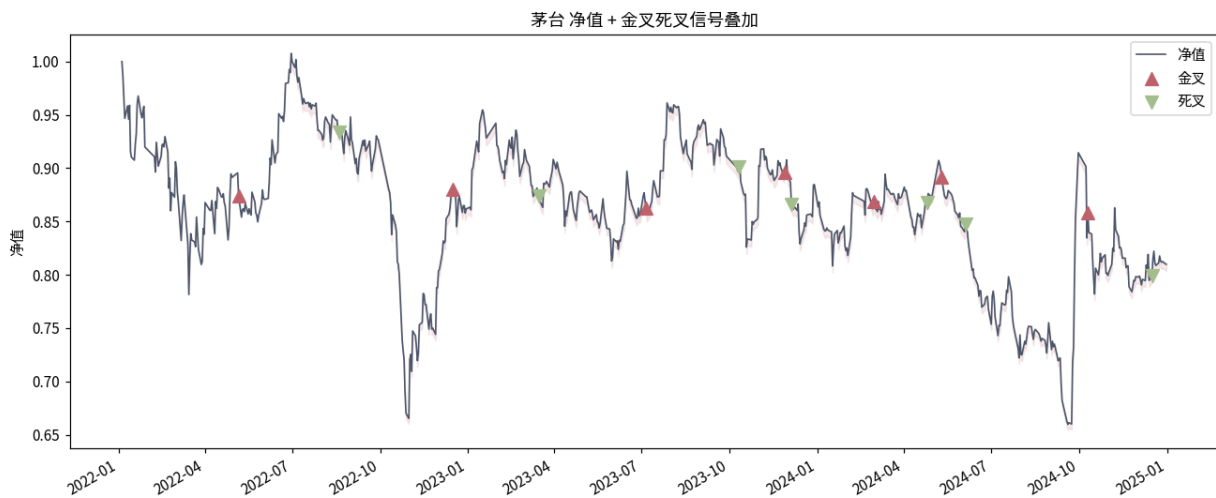
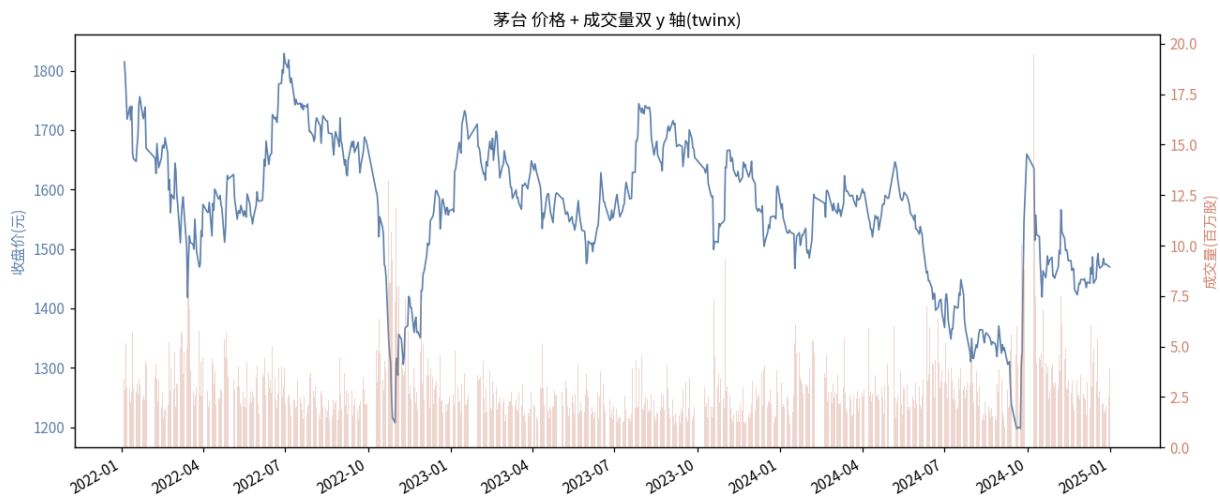
print('\n[done] 4 张图已保存,output.txt 见上方全部打印')
```

05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





绘图数据(茅台
2022-2024)

726 根日线;净值期末 0.81
倍(两年跌约两成),最大回撤
-34.5%(谷底
2024-09-19),金叉 7 次、
死叉 7 次

plt 风格 vs ax 风格

plt.plot 偷懒风格画一张简
单图够用;fig, ax =
plt.subplots() 面向对象
风格,每个 ax 是独立画框,多
子图必须用它

subplots 三面板

subplots(3,1,
sharex=True) 上中下三块
共享时间轴:价格+双均线+金
叉死叉 / 成交量+twinx净
值 / 回撤填充

twinx 双 y 轴

价格几百元、成交量几百万
股,量纲悬殊;twinx 叠加共
享 x 轴的第二个 y 轴,左右各
管一个,小的不被压扁

mdates 日期格式化

MonthLocator(interval=
3) 每三个月一个刻度 +
DateFormatter('%Y-%m')
显示成年-月,再旋转 30 度,
横轴清爽不挤成团

中文字体

run_lab 已注入 Noto Sans
CJK 中文字体 +
axes.unicode_minus=False,
中文不变方块、负号正常

ANALYSIS · 这张图告诉我们什么

► 01 一张专业图 = 多面板 + 双轴 + 干净日期轴的合力

同样一份茅台数据,小钱第一版把价格和成交量塞一个轴、日期挤成黑团,导师看不懂。改用 subplots 把价格、成交量、回撤分3块上下对齐,twinx 给量纲悬殊的量各放一个轴,mdates 把日期收拾成清爽的年-月,再把金叉死叉用散点标在价格上——一张让人一眼信服的量化分析图就成了。作图能力,直接决定你的研究能不能被别人看懂和采纳。

▶ 02 **plt 偷懒风格 vs ax 面向对象风格:多图必须用 ax**

plt.plot 直接在"当前图"上画,快速看一眼够用;但一旦要画多个子图、复杂布局,就分不清画到哪块了。fig, ax = plt.subplots() 把每个子图作为独立画框对象,想画哪块对哪个 ax 下命令,这是专业作图的地基。新手最该改的习惯,就是从 plt 偷懒风格切到 ax 风格。

▶ 03 **twinx 解决量纲悬殊:价格和成交量同框不打架**

价格几百元、成交量几百万股,直接画一个 y 轴上,成交量的大数字会把价格曲线压成贴底的直线、啥也看不出。twinx 叠加一个共享 x 轴的第二个 y 轴,左轴管价格、右轴管成交量,两个量纲悬殊的量同框展示又互不干扰,这是量价分析图的标配。

▶ 04 **回撤填充图:一眼看清最惨亏了多少、套了多久**

底部那块红色的回撤填充,深度直接告诉你最大回撤探到 -34.5%(三成四),宽度告诉你套了多久。这种填充图比单看一个回撤数字直观得多,是策略汇报里展示风险最有说服力的1 块,配合上面的净值和信号,好坏一目了然。

▶ 05 **mdates 与中文字体:专业图的两个不起眼细节**

时间轴不格式化会挤成一团黑,mdates 定好刻度间隔和格式、再旋转标签就清爽了;中文不设字体会变方块、负号会显示异常,开头注入中文字体并关掉 unicode 负号就解决。这两个细节看着小,却是一张图显不显专业的分水岭。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

量化作图

策略汇报标配:净值曲线 + 回撤面板 + 买卖信号叠加,一张图说清策略好坏

量化作图

看个股用价格 + 成交量双轴图,量价配合一眼看清

量化作图

多因子研究用 subplots 把几个因子的分层净值并列对比

量化作图

风控用回撤填充图直观展示最大回撤的深度和持续时间

量化作图

研报里用统一配色和 mdates 规范的日期轴,让图表专业可信

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

plt 和 ax 风格混用

多子图时还用 plt.plot,分不清画到了哪块画框上,图全乱。多图一律用 ax。

△ 02

量纲悬殊塞一个 y 轴

价格和成交量画一个轴,小的被压成直线。用 twinx 分两个 y 轴。

△ 03

日期轴挤成黑团

不格式化时间轴,日期标签重叠成一团。用 mdates 定刻度间隔和格式,并旋转标签。

△ 04

忘设中文字体

中文显示成方块、负号显示异常。开头必须指定中文字体并关掉 unicode 负号。

△ 05

忘了 tight_layout 或保存前 close

子图标题重叠、内存里图越堆越多。画完 tight_layout 调整间距,保存后 close 释放。

matplotlib 量化作图实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 多图一律用 fig/ax — 只有快速看一眼才用 plt 偷懒风格。
- 02 量纲悬殊用 twinx — 价格与成交量等分两个 y 轴。
- 03 时间轴必用 mdates — 定刻度间隔与格式,旋转标签防重叠。
- 04 开头设中文字体 — 指定字体并关 unicode 负号。
- 05 子图共享 x 轴对齐时间 — subplots 设 sharex,多面板时间对得齐。
- 06 画完 tight_layout 再保存 — 调间距、存图后 close 释放内存。
- 07 信号用散点叠加 — 金叉死叉等事件用 scatter 标在曲线上,一眼可见。

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

01 plt 偷懒风格快速看图,fig/ax 面向对象风格才能搭多图

02 subplots 开画框网格,sharex 让多面板时间对齐

03 twinx 给量纲悬殊的两个量各放一个 y 轴

04 mdates 格式化日期轴,定间隔、定格式、旋转标签

05 开头必设中文字体并关掉 unicode 负号

06 金叉死叉等信号用 scatter 叠加到曲线上

07 画完 tight_layout 调间距,存图后 close 释放

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 plt 风格和 fig/ax 风格的区别,什么时候必须用 ax?(提示:plt 直接在当前图上画,适合快速看一眼;fig, ax = plt.subplots() 把每个子图作为独立画框对象操作。多子图、复杂布局时必须用 ax,否则分不清画到了哪块。)

Q02 什么时候需要 twinx?(提示:当一张图里两个量的量纲相差很大时,比如价格几百、成交量几百万,画在同一个 y 轴上小的会被压扁。twinx 叠加一个共享 x 轴的第二个 y 轴,左右各管一个量。)

Q03 为什么 matplotlib 画中文常常出方块,怎么解决?(提示:因为默认字体不含中文字形。需要在开头指定支持中文的字体,并把 axes.unicode_minus 设为 False 让负号正常显示。)

Q04 多面板图怎么让几个子图的时间轴对齐?(提示:用 subplots 创建时设 sharex=True,几个子图共享同一条 x 轴,时间刻度就对得整整齐齐,上下对比同一时刻的价格、成交量、回撤。)

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ matplotlib 官方文档《The Lifecycle of a Plot》(2024)– 讲清 figure/axes 对象模型,plt vs ax 的官方说明
- ▶ matplotlib 官方文档《Date tick labels》(2024)– mdates 日期轴格式化标准参考
- ▶ Wes McKinney《Python for Data Analysis》(2022, O'Reilly)– 第九章 matplotlib 与 pandas 绘图实务
- ▶ Nicolas Rougier《Scientific Visualization: Python + Matplotlib》(2021, 开源)– 进阶作图与排版,提升图表专业度
- ▶ Edward Tufte《The Visual Display of Quantitative Information》(2001, Graphics Press)– 数据可视化的审美与原则,做出可信图表

NEXT LECTURE · 下一节预告

DAY 041 plotly 交互图

第41课我们进入一个新阶段,把前面学的数据处理本事用到真实的策略评估上,讲清楚怎么用一套标准指标全面衡量一个策略的好坏,而不是只盯着收益率。